Day 16: Linux Processes and umask

Short notes: terms, syntax, copy-paste commands and the output students will see

How to use: Paste the dark **COPY AND PASTE** boxes into a terminal on a practice machine (they use only `sleep` and `yes`). The **Output** box shows what appears; **<--** arrows explain it. PIDs and times will differ on your machine. Use a fresh terminal per topic. All commands are also in **Day16_Commands_Copy_Paste.txt**, because copying from a PDF can break lines.

1. Process, PID and PPID

Term	Simple meaning
Process	A program that is running.
PID	Process ID: the number Linux gives each process.
PPID	Parent PID: the number of the process that started it. PID 1 (systemd) is the ancestor of all.

Syntax	Meaning
`echo $$ / echo $PPID`	PID of your shell / PID of its parent.
`echo $!`	PID of the last background command.
`ps -o pid,ppid,cmd -p PID`	Show PID, parent PID and command.

COPY AND PASTE: PID and PPID

```
echo $$
echo $PPID
sleep 120 &
ps -o pid,ppid,cmd -p $$,$!
```

Output (what students will see)

```
5071                    <-- PID of my shell
5060                    <-- PID of my shell's parent
  PID  PPID CMD
 5071  5060 -bash          <-- my shell
 5200  5071 sleep 120     <-- its PPID 5071 = my shell (parent)
```

2. Types of Processes and Jobs

Type	Meaning	Example
Foreground	Holds the terminal until it ends.	sleep 5
Background	Runs while you keep working. Start with **&**.	sleep 300 &
Daemon	Always running, no terminal (TTY shows ?). Name ends with d.	sshd, crond

Type	Meaning	Example
Zombie	Finished, but its parent has not read its result. Shows Z.	(bug sign)
Orphan	Its parent died; PID 1 adopts it.	(automatic)
Job	A process started from **your shell**; has a job number [1], [2].	jobs

Syntax	Meaning
`command &`	Run in the background.
`jobs -l`	List jobs of this shell, with PIDs.
`bg %N / fg %N`	Resume job N in the background / foreground.
`Ctrl+Z / Ctrl+C`	Pause / stop the foreground job.

COPY AND PASTE: Jobs

```
sleep 300 &
sleep 400 &
jobs -l
kill %1
sleep 1
jobs
kill %2
```

Output (what students will see)

```
[1] 4821                <-- job 1 started, PID 4821
[2] 4822                <-- job 2 started, PID 4822
[1]- 4821 Running   sleep 300 &
[2]+ 4822 Running   sleep 400 &
[1]-  Terminated    sleep 300    <-- job 1 killed
```

COPY AND PASTE: Pause and resume (interactive) (interactive: follow the note)

```
sleep 600
# press Ctrl+Z  (job pauses: Stopped)
jobs
bg %1
# runs in the background now
fg %1
# press Ctrl+C  (ends it)
```

COPY AND PASTE: Daemon and zombie

```
ps -eo pid,ppid,tty,cmd | grep ' ?' | head -3
(sleep 1 & exec sleep 60) &
sleep 3
ps -eo pid,ppid,stat,cmd | grep -E '^ *[0-9]+ +[0-9]+ +Z'
```

Output (what students will see)

```
    1     0 ?        /sbin/init       <-- TTY ? = daemon, no terminal
17315 17314 Z    [sleep] <defunct>   <-- zombie, parent 17314
```

3. Process States

Code	State	Meaning in simple words
R	Running	On the CPU now, or ready and waiting for its turn.
S	Sleeping	Waiting for something (key press, network, timer). Most processes.
D	Uninterruptible sleep	Waiting for disk or hardware. Cannot be killed.
T	Stopped	Paused (Ctrl+Z or SIGSTOP).
Z	Zombie	Finished, parent has not collected the result.

About 'pending': Linux has no state called 'pending'. **Pending / waiting** means **R** (ready for a CPU turn) or **S** (waiting for an event).

Syntax	Meaning
`ps -o pid,stat,cmd -p PID1,PID2`	Show the state of several processes.
`kill -STOP PID / kill -CONT PID`	Pause / continue a process.

COPY AND PASTE: See R, S and T

```
yes > /dev/null &
RUN=$!
sleep 300 &
SLP=$!
sleep 300 &
STP=$!
sleep 1
kill -STOP $STP
sleep 1
ps -o pid,stat,cmd -p $RUN,$SLP,$STP
kill -CONT $STP
kill $RUN $SLP $STP
```

Output (what students will see)

```
  PID STAT CMD
 6101 R    yes            <-- R: running, using the CPU
 6103 S    sleep 300      <-- S: sleeping, waiting
 6105 T    sleep 300      <-- T: stopped (paused by kill -STOP)
```

4. ps: ps -ef, ps -elf, ps aux

ps shows a **snapshot** of processes. **-e** = every process, **-f** = full format, **-l** = long format (state, priority, nice), so **ps -elf** = -e -l -f. **ps aux** has **no dash**: a = all users, u = user format, x = also processes without a terminal. Teach **ps aux** (**ps -aux** usually works but mixes two styles).

Command	Shows	PPID?
ps -ef	Every process: UID, PID, PPID, start time, command	Yes
ps -elf	As -ef plus state (S), priority (PRI), nice (NI)	Yes
ps aux	Every process with %CPU, %MEM, state	No

COPY AND PASTE: ps variations

```
ps -ef | head -4
ps -elf | head -4
ps aux | head -4
ps -ef | grep '[b]ash'
ps -o ppid= -p $$
ps aux --sort=-%cpu | head -4
```

Command	What it does (in simple words)
`ps -ef | head -4`	All processes, full format (head -4 = first 4 lines).
`ps -elf | head -4`	Same plus S, PRI, NI columns.
`ps aux | head -4`	%CPU, %MEM and STAT, no PPID.
`ps -ef | grep '[b]ash'`	Find a process by name ([b] stops grep matching itself).
`ps -o ppid= -p $$`	Only the parent PID of your shell.
`ps aux --sort=-%cpu | head -4`	Top CPU users (- = biggest first).

Output of ps -ef

```
UID        PID  PPID  C STIME TTY          TIME CMD
root         1     0  0 Oct05 ?        00:00:12 /sbin/init <-- PPID 0
student   5071  5060  0 10:00 pts/0    00:00:00 -bash      <-- parent 5060
```

Output of ps -elf (some columns hidden)

```
F S UID       PID  PPID  C PRI  NI STIME TTY       TIME CMD
0 S student  5071  5060  0  80   0 10:00 pts/0 00:00:00 -bash <-- S=sleeping
0 R student  5120  5071 99  80   0 10:25 pts/0 00:00:45 yes   <-- R=running
```

5. top: Live Monitoring

Syntax	Meaning
`top`	Live view, refreshed every 3 seconds. Press q to quit.
`top -u USER`	Only one user's processes.
`top -b -n 1`	One snapshot, then exit (good for pasting).

COPY AND PASTE: Create load, then watch it in top (interactive: follow the note)

```
yes > /dev/null &
nproc
uptime
top
top -b -n 1 | head -12
```

Command	What it does (in simple words)
`yes > /dev/null &`	Starts a process that uses one full CPU core.
`nproc ; uptime`	Number of cores; load average. Load above the core count = overloaded.
`top`	Live view. Press q to quit.
`top -b -n 1 | head -12`	Same as one text snapshot.

Output (what students will see)

```
top - 10:30:15 up 5 days, load average: 0.52, 0.48, 0.45 <-- load 1/5/15 min
Tasks: 210 total,   1 running, 208 sleeping, 1 zombie <-- by state
%Cpu(s):  5.0 us, 2.0 sy, 92.0 id, 1.0 wa <-- id=idle wa=disk wait
MiB Mem :  7900.0 total,  3500.0 free,  2100.0 used

  PID USER     PR  NI S %CPU %MEM COMMAND
 5120 student  20   0 R 99.7  0.0 yes   <-- uses the CPU
```

Keys in top: P sort by CPU, **M** sort by memory, **k** kill (type PID), **r** renice (type PID), **1** show each core, **q** quit.

6. htop: Friendlier top

Syntax	Meaning
`htop`	Start htop (F10 or q quits).
`htop --tree`	Start in tree view (shows parent and child).
`sudo apt install -y htop`	Install on Ubuntu/Debian. Red Hat: sudo dnf install -y htop.

COPY AND PASTE: Install and open htop (interactive: follow the note)

```
sudo apt install -y htop
htop --tree
```

Key	Action
`F3 or /`	Search a process by name
`F5 or t`	Tree view (parent and child)
`F7 / F8`	Nice - (higher priority, needs sudo) / Nice + (lower priority)

Key	Action
`F9 or k`	Kill: choose a signal, press Enter
`F10 or q`	Quit

top vs htop: top is installed everywhere and uses typed PIDs. htop has colours, mouse support and a tree view, and uses function keys, but is usually not installed by default.

7. nice and renice: Priority

The **nice value** decides who gets the CPU first. **Higher number = lower priority.** Range **-20** (highest, root only) to **19** (lowest). Default **0**.

Syntax	Meaning
`nice -n VALUE command`	Start a command with this nice value.
`renice -n VALUE -p PID`	Change a running process.
`sudo renice -n -5 -p PID`	Raise priority (negative needs sudo).

COPY AND PASTE: nice and renice

```
nice -n 10 sleep 300 &
sleep 1
ps -o pid,ni,cmd -p $!
renice -n 15 -p $!
ps -o pid,ni,cmd -p $!
sudo renice -n -5 -p $!
ps -o pid,ni,cmd -p $!
kill $!
```

Output (what students will see)

```
  PID  NI CMD
 5300  10 sleep 300     <-- started with nice 10
5300 (process ID) old priority 10, new priority 15
  PID  NI CMD
 5300  15 sleep 300     <-- renice made it nicer (lower priority)
  PID  NI CMD
 5300  -5 sleep 300     <-- sudo raised its priority
```

8. The kill Command

kill sends a **signal** to a process, chosen by its PID. A signal can mean 'please stop', 'stop now', 'pause' or 'continue'. **Without a signal, kill sends 15 (polite stop).**

Syntax	Meaning
`kill PID`	Polite stop (signal 15, SIGTERM).
`kill -9 PID`	Force stop (SIGKILL). Last resort.
`kill -STOP PID / kill -CONT PID`	Pause / continue.

Syntax	Meaning
`kill -l`	List all signals.
`kill %N`	Kill job number N.

Signal	No.	Meaning
SIGTERM	15	Please stop. Program can clean up first. **Try this first.**
SIGKILL	9	Stop at once. Cannot be ignored. No cleanup.
SIGHUP	1	Hang up; many services reload settings.
SIGINT	2	Same as Ctrl+C.
SIGSTOP / SIGCONT	19 / 18	Pause / continue.

COPY AND PASTE: kill step by step

```
sleep 300 &
P1=$!
sleep 300 &
P2=$!
sleep 300 &
P3=$!
ps -o pid,stat,cmd -p $P1,$P2,$P3
kill $P1
kill -STOP $P2
sleep 1
ps -o pid,stat,cmd -p $P1,$P2,$P3
kill -9 $P2 $P3
sleep 1
ps -o pid,stat,cmd -p $P1,$P2,$P3
kill -l | head -3
```

Command	What it does (in simple words)
`sleep 300 & ; P1=$! (x3)`	Three practice processes; each PID saved in a variable.
`ps -o pid,stat,cmd -p ...`	Show all three: all S (sleeping).
`kill $P1`	Polite stop of the first one.
`kill -STOP $P2`	Pause the second one.
`kill -9 $P2 $P3`	Force-kill the other two (works even on a stopped process).
`kill -l | head -3`	Show the first lines of the signal list.

Output (what students will see)

```
  PID STAT CMD
 6201 S    sleep 300
 6202 S    sleep 300
 6203 S    sleep 300         <-- all three sleeping
  PID STAT CMD
 6202 T    sleep 300     <-- P2 stopped (P1 is gone)
 6203 S    sleep 300     <-- P3 still sleeping
  PID STAT CMD               <-- heading only: all gone
```

When kill (15) is ignored, use kill -9

COPY AND PASTE: A stubborn process

```
bash -c 'trap "" TERM; while true; do sleep 1; done' &
BAD=$!
sleep 1
kill $BAD
sleep 2
ps -o pid,stat,cmd -p $BAD
kill -9 $BAD
sleep 1
ps -o pid,stat,cmd -p $BAD
```

Output (what students will see)

```
  PID STAT CMD
 6310 S    bash -c trap "" TERM; ...   <-- alive after kill
  PID STAT CMD                         <-- kill -9 removed it
```

Careful: You can kill only **your own** processes (others need sudo). Never kill PID 1 or unknown system processes. Order: **kill PID**, wait, check with ps, then **kill -9 PID**. A zombie cannot be killed: stop its **parent**. Find a PID with `ps -ef` or `echo $!`.

9. umask: Default Permissions of New Files

umask decides which permissions are **removed** from every new file and folder. New files start from **666** (rw-rw-rw-), folders from **777** (rwxrwxrwx), and the umask takes permissions away.

umask	New file	New folder	Meaning
022	644 (rw-r--r--)	755 (rwxr-xr-x)	Group and others cannot write. Common default.
002	664 (rw-rw-r--)	775 (rwxrwxr-x)	Group can write too. Ubuntu users default.
027	640 (rw-r-----)	750 (rwxr-x---)	Others get nothing.
077	600 (rw-------)	700 (rwx------)	Private: only the owner.

Calculation: file = 666 minus umask, folder = 777 minus umask. Example: umask 022 gives file 644 and folder 755. New files never get **x** automatically (use chmod +x).

Syntax	Meaning
umask / umask -S	Show the mask in numbers / in letters.
umask MODE	Set it for this terminal only, e.g. umask 077.
echo 'umask 027' >> ~/.bashrc	Make it permanent for your login shell.

COPY AND PASTE: See umask change new files

```
umask
mkdir -p ~/umask_lab && cd ~/umask_lab
touch file_a
mkdir dir_a
umask 077
touch file_b
mkdir dir_b
umask 002
touch file_c
mkdir dir_c
ls -ld file_? dir_?
umask 022
cd ~ && rm -r ~/umask_lab
```

Output (what students will see)

```
0022                                                <-- current mask
drwxr-xr-x 2 student student 4096 Oct 13 11:37 dir_a   <-- mask 022: 755
drwx------ 2 student student 4096 Oct 13 11:37 dir_b   <-- mask 077: 700
drwxrwxr-x 2 student student 4096 Oct 13 11:37 dir_c   <-- mask 002: 775
-rw-r--r-- 1 student student    0 Oct 13 11:37 file_a  <-- 644
-rw------- 1 student student    0 Oct 13 11:37 file_b  <-- 600
-rw-rw-r-- 1 student student    0 Oct 13 11:37 file_c  <-- 664
```

10. Cleanup and Cheat Sheet

COPY AND PASTE: Clean up after the demos

```
pkill -u $USER yes
pkill -u $USER sleep
kill $(jobs -p) 2>/dev/null
jobs
```

Command	Meaning
ps -ef / ps -elf / ps aux	All processes: PID+PPID / plus state, nice / %CPU, %MEM
top (P M k r q) / htop (F3 F5 F7 F8 F9)	Live monitoring; sort, kill, renice
nice -n 10 cmd / renice -n 15 -p PID	Start / change priority
kill PID / kill -9 PID / kill -STOP PID	Polite stop / force stop / pause

Command	Meaning
`cmd &, Ctrl+Z, jobs, bg, fg`	Background, pause, list, resume
`umask / umask 027`	Show / set default permission mask